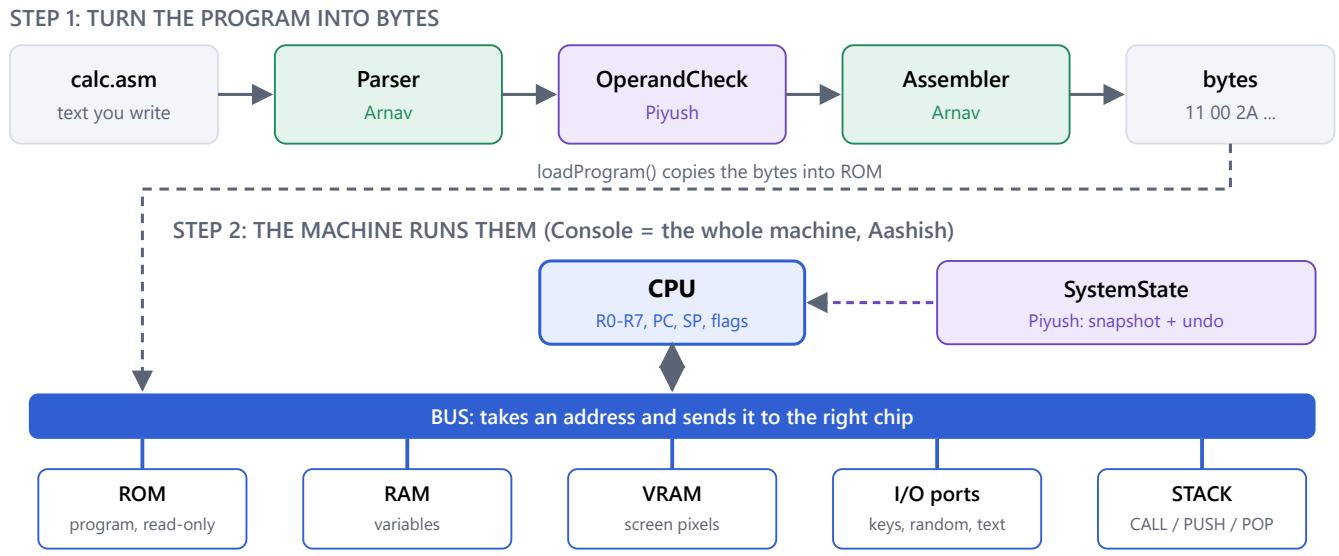
PIXEL-8: How Our CPU Works

The easy version: what each part does, how they connect, how the calculator uses the CPU, and which C / C++ ideas are used where.

> **In one line**
> We built a small pretend computer in C++. It has a CPU, memory and a "bus" (the wires between them). You write a program in our assembly language, our tools turn it into bytes, and the CPU runs those bytes one instruction at a time, just like a real 8-bit chip.

1 The big picture



Simple way to say it: the CPU is the brain, memory is the notebook, and the bus is the postman. The CPU never touches a memory chip directly. It just tells the bus "give me address `0x8000`", and the bus works out which chip owns that address.

Who owns which address (the memory map)

Addresses	Chip	What it is for
`0x0000 – 0x7FFF`	ROM (32 KB)	The program. Read-only, like a printed book. Writing here is an error.
`0x8000 – 0xDFFF`	RAM (24 KB)	Variables. The calculator keeps its numbers and answer here.
`0xE000 – 0xEFFF`	VRAM (4 KB)	The screen: 32 × 16 pixels, one byte per pixel.
`0xF000 – 0xF0FF`	I/O ports	Buttons, a random number maker, a frame counter, and text output.
`0xF100 – 0xFFFF`	Stack	Return addresses for CALL, and values saved with PUSH.

2 How the CPU works

The CPU repeats the same three steps forever. This is called the **fetch, decode, execute** cycle (`CPU::step()`):

1. **Fetch:** read the next byte from memory at the address in **PC** (program counter), then move PC forward.
2. **Decode:** look that byte up in the opcode table to find out which instruction it is (for example `0x30` means `ADD`).
3. **Execute:** do it, for example add two registers and update the flags.

What is inside the CPU

Part	Easy meaning
R0 – R7	8 tiny boxes, each holds one number from 0 to 255
PC	"Which line am I on?" (address of the next instruction)
SP	"Where is the top of the stack?"
R6:R7 = P	Two registers together hold one 16-bit address, which works like a C pointer

The 4 flags (yes/no notes)

Flag	Turns on when...
Z zero	the answer was 0
C carry	the answer went over 255 or below 0
N negative	the top bit is 1 (a negative number)
V overflow	a signed answer came out wrong

Jumps like `JZ` and `JC` look at these flags. That is how the CPU makes decisions, the same as an `if` in C.

> **It never crashes**
> When a program does something wrong (writes into ROM, uses register R9, jumps outside the program, overflows the stack), the CPU does not crash. It stops and records a **Fault** that says what went wrong and where.

3 How each unit connects to the CPU

Unit	Who	What it does	How it connects
Parser	Arnav	Reads the `.asm` text and splits each line into the instruction name, the operands and the line number. Removes comments.	Gives a list of `Instruction` to the checker and assembler
OperandCheck	Piyush	Catches mistakes before running, for example `LDI R0, 300` (too big) or a label that does not exist. Gives the line number.	Reads the CPU's own opcode table, so it always agrees with the CPU
Assembler	Arnav	Turns the checked instructions into the bytes the CPU understands. Works out where each label is.	Its bytes go into ROM through `Console::loadProgram()`
SystemState	Piyush	Takes a snapshot of the CPU (registers, flags, PC, SP) and can put it back, which is used for undo.	Uses the CPU's getters and setters (`getReg` , `setPC` , ...)
CPU, Bus, memory, Console	Aashish	The machine itself. `Console` builds every chip and wires it to the bus at the right address.	Everyone plugs into `Console`

4 How the calculator uses the CPU

The calculator is a program written in **our own assembly language** (`CPU/programs/calc.asm`). The C++ side only passes the numbers in and reads the answer out. **The actual maths is done by our CPU.**

1. When it starts, **Parser** → **OperandCheck** → **Assembler** turn `calc.asm` into 164 bytes, and these go into **ROM**.
2. You type `25 + 17`. The C++ side writes 25 into RAM address `0x8000` and 17 into `0x8001`, **through the bus**.
3. It sets the CPU's PC to the start of the add routine (the label `do_add`, found by the assembler) and lets it run.
4. The CPU runs until `HLT` (stop). The C++ side then reads the answer from RAM address `0x8002`.

```
> 25 + 17
  LD   R0, [0x8000]   load the first number from RAM       R0 = 25
  LD   R1, [0x8001]   load the second number from RAM      R1 = 17
  ADD  R0, R1         add them                             R0 = 42
  ST   R0, [0x8002]   save the answer to RAM
  HLT                 stop
= 42          5 instructions run on the CPU
```

Multiply and divide are loops

Our CPU has no multiply or divide instruction, and neither did many real 8-bit chips. So `7 * 6` is done as "add 7, six times", and `50 / 7` as "take away 7 until you can't". The calculator shows how many times each line ran:

```
  ADD  R2, R0    x6   add the first number to the answer     R2 = 42
  DEC  R1        x6   one fewer time to go
  JNZ  mul_loop  x6   repeat until done          <- like a while loop in C
```

Showing the 8-bit limit

You type	CPU says	Why
`200 + 100`	44	300 does not fit in 8 bits (max 255), so it wraps: 300 − 256 = 44. The **C flag** turns on.
`5 - 9`	252	It went below zero. 8 bits store −4 as 252 (256 − 4). The **C** and **N** flags turn on.
`9 / 0`	error	The program checks for 0 first and stops safely, with no crash.

> **How we know it is right**
> The test `test_calc.cpp` runs **every possible pair** of numbers (256 × 256 = 65,536 pairs) through the CPU for + − AND OR XOR and compares each answer with C++. All of them match. Altogether there are 287 tests across the project, and all pass.

5 C / C++ ideas we used, and where

Idea	Where in our code	In easy words
Stack (last in, first out)	CPU stack at `0xF100–0xFFFF`: `PUSH`, `POP`, `CALL`, `RET`	Like a pile of plates: you take off the last one you put on. `CALL` puts the return address on the pile, and `RET` takes it off to get back. A C++ function call works the same way. Our stack grows *downward* from `0xFFFF`. Pushing onto a full stack or popping an empty one gives a Fault.
Recursion (uses the C++ call stack)	`Parser`: `parseLine()` calls itself	For `loop: DEC R0`, it saves the label and then calls itself again on the rest of the line.
Queue (first in, first out)	*Not used yet*	Nothing in the CPU needs one today. A good place later would be a keyboard buffer, so key presses are handled in the order they happened.
Array / lookup table	Opcode table with 256 slots, and registers `Byte reg[8]`	To decode, the CPU uses the byte as an index: `slot[0x30]` gives ADD straight away, with no searching (O(1)).
Pointer	The Bus stores `Device*`. CPU pair `R6:R7` = P, with `LDP` / `STP` / `INP`	A pointer is an address. `LDP R0` means `R0 = *P`, and `INP` means `P++`, just like walking through an array in C.
Reference (`&`)	`CPU(Bus& bus)`, and `pop(Byte& value)`	The CPU keeps a reference to the one shared bus, not a copy, so everyone talks to the same bus.
Inheritance + virtual (polymorphism)	`Device` base class. `Ram`, `Rom` and `IoPorts` inherit from it	Every chip has `read()` and `write()`. The bus calls `device->read()` without caring which chip it is, and C++ picks the right version. ROM's `write()` refuses, which is how ROM is read-only.
std::vector (growing array)	RAM/ROM bytes, the bus's list of chips, the assembler's output bytes, the parsed lines	An array that can grow. The bus finds a chip by checking each entry in its list of chips.
std::map (key → value)	Assembler: label → address and address → source line. Calculator: line → times run	Like a dictionary. "Where is `mul_loop`?" gives its address.
std::set (no duplicates)	`OperandCheck` label list	If a label is added twice, the set says so, and we report "label defined twice".
struct	`Instruction`, `Fault`, `SystemState`, `CalcResult`	Groups related values together. For example a Fault has a type, an address and a message.
enum class	`CpuState` (Running/Halted/Faulted), `FaultType`, `Format`	Named choices instead of magic numbers, so the code reads like English.
typedef	`Byte` = `uint8_t`, `Word` = `uint16_t`	Short names: Byte is 0–255 (data), Word is 0–65535 (addresses).

Bitwise operators `& | ^ << >>`	Flags packed into one byte. Assembler packs two registers into one byte: `(dst<<4) | src`	Each flag is one bit of a byte. 16-bit addresses are split into a low byte and a high byte (stored low byte first, called *little-endian*).
switch	`CPU::execute()` and the Assembler	One `case` per instruction: "if it's ADD, do this; if it's JMP, do that".
Callback (`std::function`)	Bus write listener, console output listener	"Call me whenever memory changes." Piyush's undo uses it to remember old values.
Operator overloading	`SystemState ==`, and `operator bool` in OperandCheck	Lets us write `if (before == after)` or `if (!check)` with our own types.
Errors: exception vs. error value	Parser throws an exception (`throw`) if the file is missing. The CPU uses a `Fault` value instead	A missing file is a real problem, so it throws. A bad program is expected sometimes, so the CPU just stops and reports it.
Two-pass algorithm	Assembler	Pass 1 finds where every label is. Pass 2 writes the bytes. This is how a jump can go to a label further down.
namespace, **stringstream**, **lambda + sort**	`MemoryMap`, the Parser, the Calculator	Keeps constants in one named group, splits text by commas, and sorts the trace lines.

6 How to run it

```
# in PowerShell, from the CPU folder
cd C:\PBLCODE\8bit-console\CPU
mingw32-make calc              # type sums yourself: 25 + 17, 7 * 6, 50 / 7 ...
mingw32-make calc ARGS=demo    # 11 ready-made examples, press Enter for each one
mingw32-make test              # run all 287 tests
```

PIXEL-8 8-bit console project · CPU, Bus, memory and Console: Aashish · Parser and Assembler: Arnav · OperandCheck and SystemState: Piyush